

더 나쁜 양자 프로그램이 더 잘 최적화 된다면?

KAIST 프로그래밍언어 연구실

이강욱 장유진 류석영

좋은 입력 → 좋은 최적화 결과?

더 좋은 소스 입력이 더 좋은 최적화를 보장하지 않음

최적화 도구:

$e + (-e) \rightarrow 0$

$e + 0 \rightarrow e$

$(x + (-x)) + 0;$



$0;$

좋은 입력 → 좋은 최적화 결과?

더 좋은 소스 입력이 더 좋은 최적화를 보장하지 않음

최적화 도구:

$e + (-e) \rightarrow 0$

$e + 0 \rightarrow e$

~~$e - e \rightarrow 0$~~

$(x + (-x)) + 0;$



$0;$

$x - x$



$x - x;$

좋은 입력 → 좋은 최적화 결과?

더 좋은 소스 입력이 더 좋은 최적화를 보장하지 않음

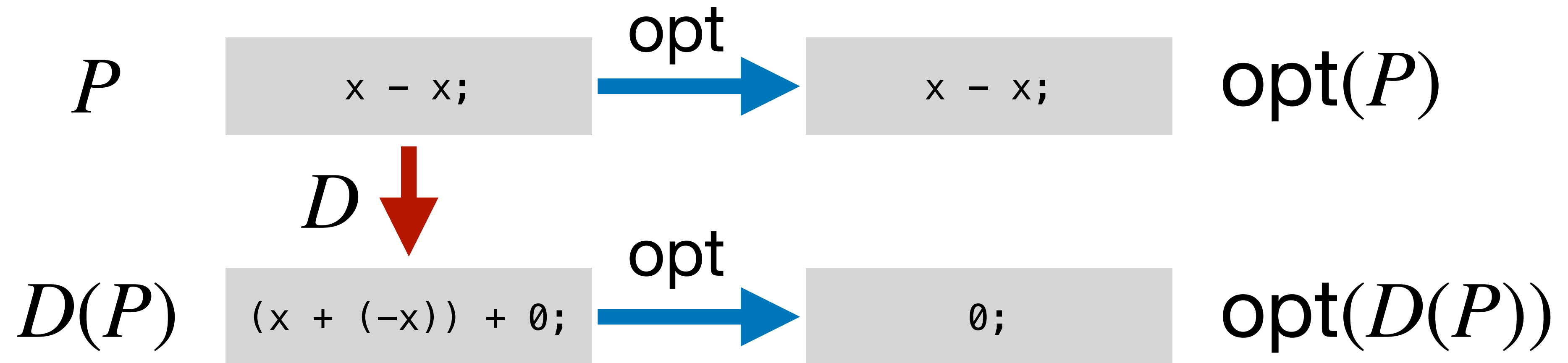
- 고전적인 컴파일러 연구 결과
 - 더 효율적인 입력이 더 비효율적인 출력 코드로 이어짐^[1]
 - 일부러 비효율화한 프로그램이 더 잘 최적화되는 사례^[2]

[1] Theodoridis and Su, “Refined Input, Degraded Output: The Counterintuitive World of Compiler Behavior,” PLDI ’24.

[2] Gao et al., “Shoot Yourself in the Foot — Efficient Code Causes Inefficiency in Compiler Optimizations,” ASE ’24

이 현상을 어떻게 테스트할까?

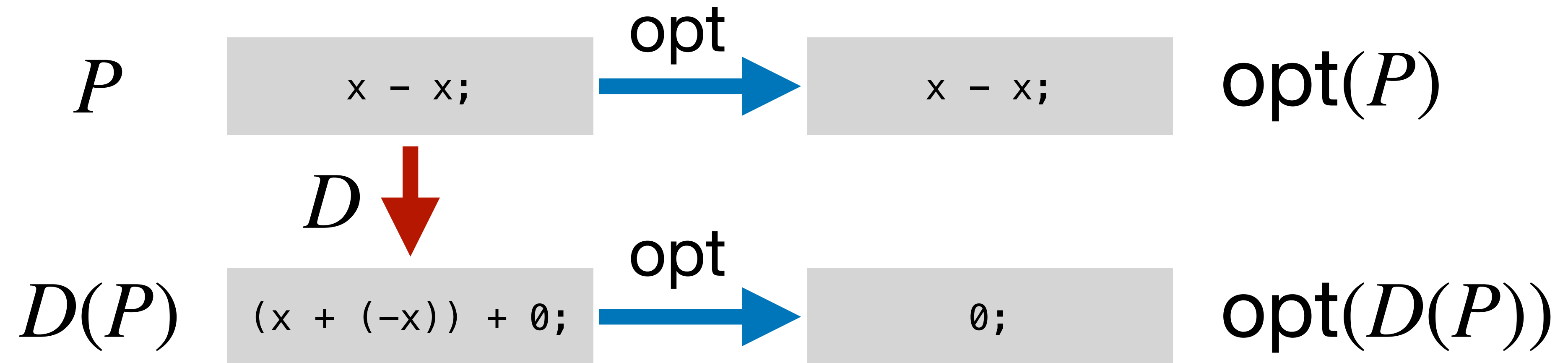
같은 예시를 역최적화 D 로 표현



- D : 프로그램 의미는 보존하면서 비용을 증가시키는 역최적화
- $\text{sem}(D(P)) = \text{sem}(P)$
- $\text{cost}(D(P)) > \text{cost}(P)$

이 현상을 어떻게 테스트할까?

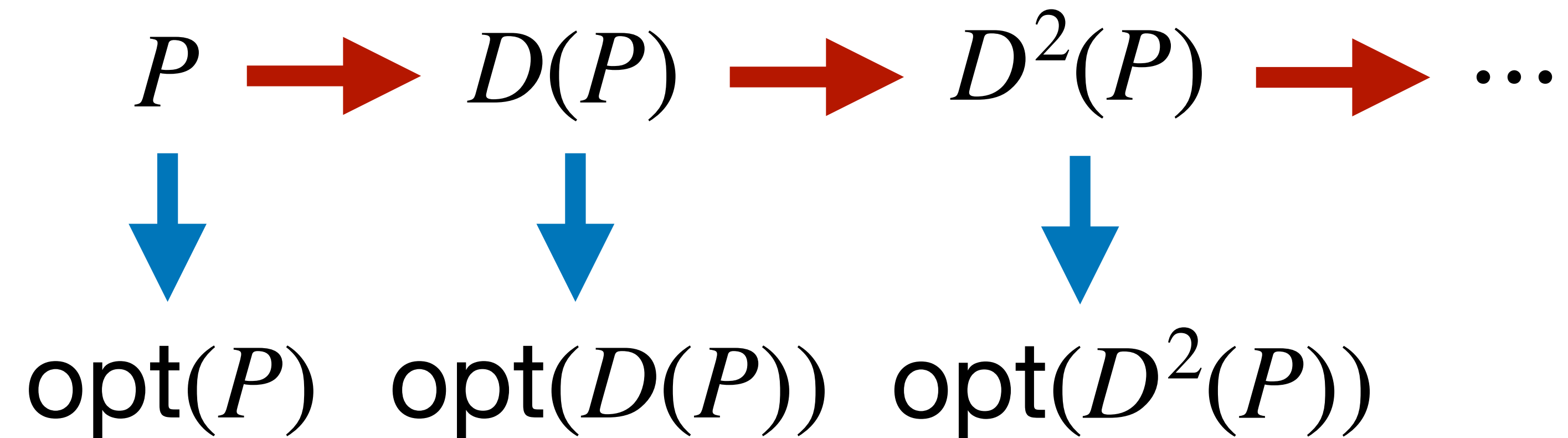
같은 예시를 역최적화 D 로 표현



- 예시: 더 나쁜 동치 입력이 더 좋은 최적화 결과를 만듦
- $\text{cost}(\text{opt}(D(P))) < \text{cost}(\text{opt}(P))$

이 현상을 어떻게 테스트할까?

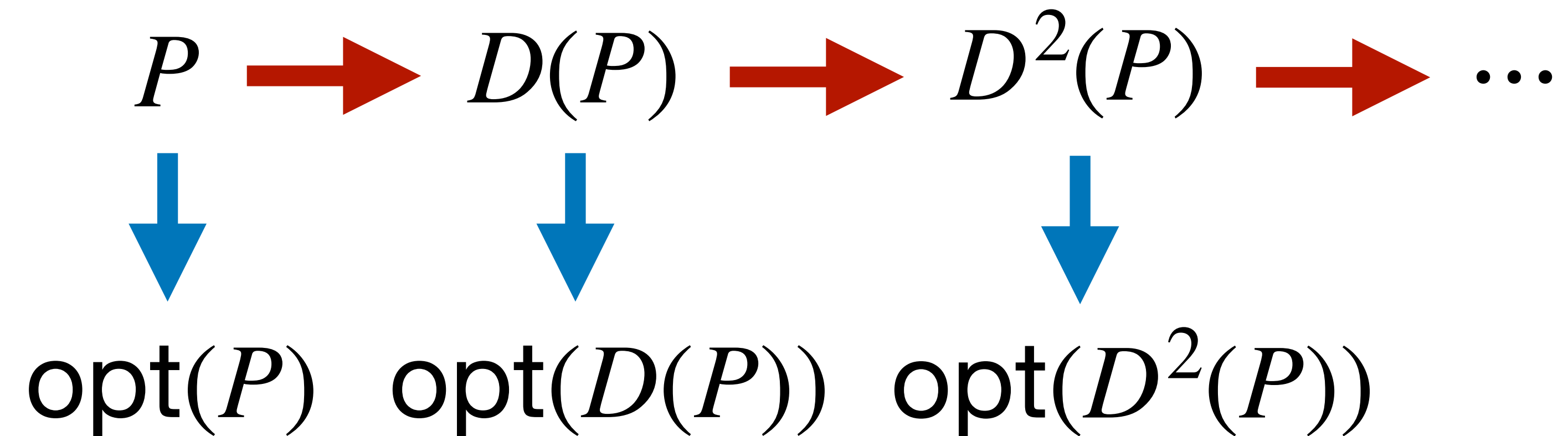
“좋은 입력” → “좋은 최적화” 테스트 프레임워크



- $D^n(P)$: 더 다양하고 복잡한 역최적화
- 비용 역전 시 “좋은 입력 → 좋은 최적화” 위반

이 현상을 어떻게 테스트할까?

고전 프로그램 적용 시 어려운 점



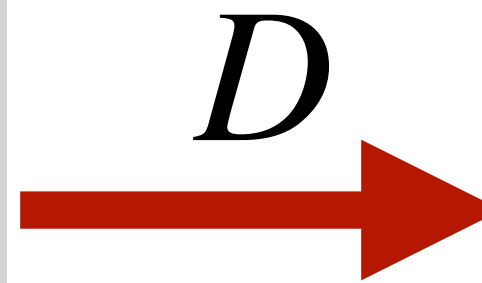
- 소스와 출력 코드의 표현이 달라 동일한 비용 함수 적용 어려움
- “더 좋음” 기준 모호 (실행 시간, 코드 크기, 메모리 접근)
- 넓은 언어에 대해 D 의 의미 보존 보장 어려움

양자 프로그램에서는?

1. 비용 함수 비교적 명확

입력 양자 프로그램

```
qreg q[1];  
creg c[1];  
  
measure q -> c;
```



역최적화된 양자 프로그램

```
qreg q[1];  
creg c[1];  
  
x q[0];  
x q[0];  
  
measure q -> c;
```

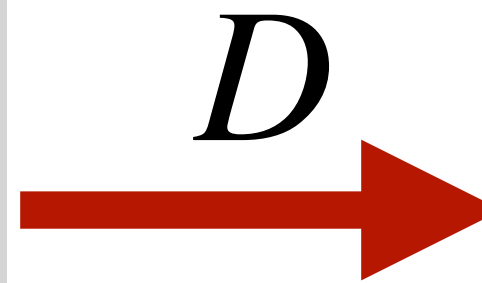
- 입력과 출력이 모두 양자 프로그램 (QASM)
- 정규화하여 양자 연산 수 비교 ($0 \rightarrow 2$)

양자 프로그램에서는?

2. 역최적화 D 의미 보존 증명 가능

입력 양자 프로그램

```
qreg q[1];  
creg c[1];  
  
measure q -> c;
```



역최적화된 양자 프로그램

```
qreg q[1];  
creg c[1];  
  
x q[0];  
x q[0];  
  
measure q -> c;
```

- 의미를 보존하는 재작성 규칙으로 D 정의 ($\text{nop} \rightarrow x; x$)
- Rocq으로 증명: $\forall P. \text{sem}(D(P)) = \text{sem}(P)$

진행 상황

의미 보존 증명

- 프로그램 의미 동치 정의

Definition Instruction_eq

(instr1 instr2: Instruction) :=

forall (ps: ProgramState),

ProgramState_invariant ps ->

ProgramState_eq (Execute instr1 ps) (Execute instr2 ps).

진행 상황

의미 보존 증명

- 역최적화 재작성 규칙 정의 및 의미 동치 증명

```
Lemma Instruction_eq_rewrite:
  forall (pre_instr post_instr: Instruction)
    (instr1 instr2: Instruction),
    Instruction_eq instr1 instr2 ->
    Instruction_eq
    qasm{ pre_instr; instr1; post_instr }
    qasm{ pre_instr; instr2; post_instr }.

Lemma Transform_X_X: forall (qbit: nat),
  qbit < nq ->
  Instruction_eq
  qasm{ X qbit; X qbit }
  qasm{ I qbit }.

Lemma Transform_swap_insert:
  forall (qbit1 qbit2: nat) (instr: Instruction),
  qbit1 < nq -> qbit2 < nq ->
  Instruction_eq
  qasm{
    swap qbit1 qbit2; $(swap_qbit_instr qbit1 qbit2 instr)
  }
  qasm{ instr }.
```

진행 상황

간이 테스트 프레임워크 개발

- 12가지 최적화기, 32,768 테스트 케이스
- 간이 테스트: Qiskit, Cirq에서만 677,428개의 원시 실패 발견
- 중복 제거, 최소화, 원인 분석 진행 중
- 다음 단계: 검증된 규칙을 테스트 생성기와 통합